

COMPTE RENDU

Administration Réseaux et Programmation Système - TP3 - Processus et
Ordonnancement
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignants : M. Noufel & M. ElAmrani
Date : 22 février 2026

Table des matières

1	Exercice 1 : Systèmes multiprogrammés et ordonnancement	2
1.1	Définition d'un système multiprogrammé	2
1.2	Principe de fonctionnement	2
1.3	Analyse des processus A et B	2
2	Exercice 2 : Algorithmes d'ordonnancement — 5 processus	3
2.1	Données	3
2.2	FCFS (First Come First Served)	3
2.3	Round Robin — quantum = 1	3
2.4	Round Robin — quantum = 4	4
2.5	SJF sans préemption	4
2.6	SRT (Shortest Remaining Time)	5
3	Exercice 3 : Algorithmes d'ordonnancement — 4 processus	5
3.1	Données	5
3.2	FCFS	5
3.3	SJF sans préemption	6
3.4	SJF avec préemption (SRT)	6
3.5	Round Robin — quantum = 2	7
3.6	Round Robin — quantum = 1	7
4	Exercice 4 : Ordonnancement par priorités	8
4.1	Données	8
4.2	Ordonnancement par priorités — sans préemption	8
4.3	Ordonnancement par priorités — avec préemption	8
	Conclusion	9

1 Exercice 1 : Systèmes multiprogrammés et ordonnancement

1.1 Définition d'un système multiprogrammé

Un système multiprogrammé (ou multitâche) est un système d'exploitation capable de maintenir plusieurs processus simultanément en mémoire centrale. L'objectif est d'optimiser l'utilisation du processeur en évitant qu'il reste inactif pendant les opérations d'entrées/sorties d'un processus.

1.2 Principe de fonctionnement

Lorsqu'un processus en cours d'exécution doit attendre une opération d'E/S (lecture disque, réseau, etc.), le système d'exploitation lui retire le processeur et l'attribue à un autre processus prêt à s'exécuter. Ce mécanisme de commutation de contexte permet d'occuper le CPU en permanence et d'améliorer les performances globales du système.

1.3 Analyse des processus A et B

D'après le diagramme du sujet, les processus A et B alternent des phases d'exécution CPU et des phases d'attente E/S.

Sans multiprogrammation : les processus sont exécutés séquentiellement. On exécute A entièrement (phases CPU + attentes), puis B entièrement. Le CPU reste inactif pendant toutes les phases d'attente E/S. Le temps total est la somme des durées individuelles de A et de B (y compris leurs temps d'attente).

Avec multiprogrammation : pendant que A attend une E/S, le CPU est donné à B. Les temps d'attente des deux processus se chevauchent. Le temps total est significativement réduit car le CPU n'est presque jamais inactif.

Politique d'ordonnancement : ensemble de règles et d'algorithmes déterminant à quel processus prêt le CPU doit être attribué à chaque instant, et pendant combien de temps.

Principales politiques utilisées :

- **FCFS** (First Come First Served) : ordre d'arrivée strict, sans préemption ;
- **SJF** (Shortest Job First) : priorité au processus dont le temps de traitement restant est le plus court, sans préemption ;
- **SRT** (Shortest Remaining Time) : variante préemptive du SJF ;
- **RR** (Round Robin) : chaque processus reçoit un quantum de temps fixe, puis est remis en file ;
- **Priorités** : le processus de plus haute priorité est exécuté en premier, avec ou sans préemption.

Critères d'évaluation :

- **Temps de rotation** (turnaround time) : date de fin – date d'arrivée ;
- **Temps d'attente** : temps de rotation – temps de traitement ;
- **Rendement** (throughput) : nombre de processus terminés par unité de temps ;
- **Utilisation du CPU** : pourcentage du temps où le CPU est actif.

Rôle d'un bon ordonnanceur : maximiser l'utilisation du CPU et le rendement, tout en minimisant les temps de rotation et d'attente, de manière équitable pour tous les processus.

2 Exercice 2 : Algorithmes d'ordonnancement — 5 processus

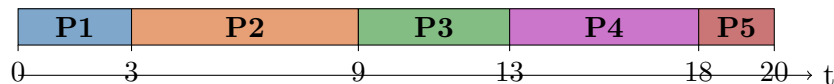
2.1 Données

Processus	Date d'arrivée	Temps de traitement
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

La durée totale d'exécution est $3 + 6 + 4 + 5 + 2 = 20$ unités de temps.

2.2 FCFS (First Come First Served)

Les processus sont exécutés dans l'ordre d'arrivée : P1, P2, P3, P4, P5.



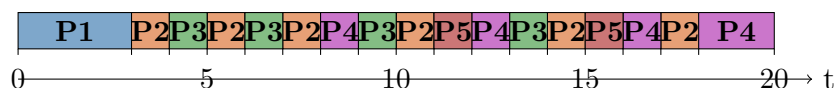
Processus	Début	Fin	Rotation	Attente
P1	0	3	$3 - 0 = 3$	0
P2	3	9	$9 - 2 = 7$	1
P3	9	13	$13 - 4 = 9$	5
P4	13	18	$18 - 6 = 12$	7
P5	18	20	$20 - 8 = 12$	10
Moyenne			$43/5 = 8.6$	$23/5 = 4.6$

Rendement : $5/20 = 0.25$ processus/unité de temps.

2.3 Round Robin — quantum = 1

Chaque processus reçoit au maximum 1 unité de temps avant d'être remis en file d'attente. L'ordre d'exécution résultant est le suivant :

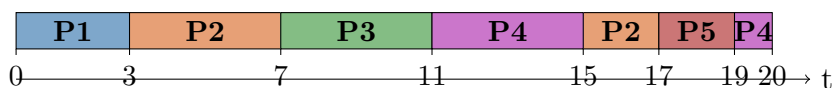
P1(0→1), P1(1→2), P1(2→3)→**fin P1 à t=3**, P2(3→4), P3(4→5), P2(5→6), P3(6→7), P2(7→8), P4(8→9), P3(9→10), P2(10→11), P5(11→12), P4(12→13), P3(13→14)→**fin P3 à t=14**, P2(14→15), P5(15→16)→**fin P5 à t=16**, P4(16→17), P2(17→18)→**fin P2 à t=18**, P4(18→19), P4(19→20)→**fin P4 à t=20**.



Processus	Fin	Rotation	Attente
P1	3	$3 - 0 = 3$	$3 - 3 = 0$
P2	18	$18 - 2 = 16$	$16 - 6 = 10$
P3	14	$14 - 4 = 10$	$10 - 4 = 6$
P4	20	$20 - 6 = 14$	$14 - 5 = 9$
P5	16	$16 - 8 = 8$	$8 - 2 = 6$
Moyenne		$51/5 = 10.2$	$31/5 = 6.2$

Rendement : $5/20 = 0.25$ processus/unité de temps.

2.4 Round Robin — quantum = 4



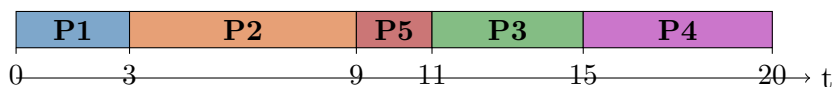
Processus	Fin	Rotation	Attente
P1	3	$3 - 0 = 3$	0
P2	17	$17 - 2 = 15$	9
P3	11	$11 - 4 = 7$	3
P4	20	$20 - 6 = 14$	9
P5	19	$19 - 8 = 11$	9
Moyenne		$50/5 = 10$	$30/5 = 6$

Rendement : $5/20 = 0.25$ processus/unité de temps.

2.5 SJF sans préemption

À chaque libération du CPU, on sélectionne parmi les processus disponibles celui dont le temps de traitement est le plus court.

- $t = 0$: seul P1 disponible $\rightarrow$ P1 (durée 3), finit à $t = 3$.
- $t = 3$: P2 est arrivé (à $t = 2$), P3 pas encore $\rightarrow$ P2 (durée 6), finit à $t = 9$.
- $t = 9$: disponibles : P3(4), P4(5), P5(2) $\rightarrow$ P5 le plus court, finit à $t = 11$.
- $t = 11$: disponibles : P3(4), P4(5) $\rightarrow$ P3, finit à $t = 15$.
- $t = 15$: P4 $\rightarrow$ finit à $t = 20$.



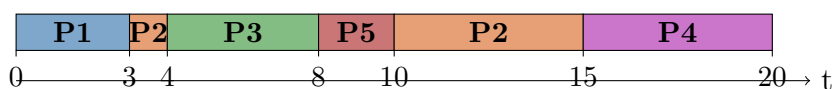
Processus	Fin	Rotation	Attente
P1	3	3	0
P2	9	7	1
P5	11	3	1
P3	15	11	7
P4	20	14	9
Moyenne		$38/5 = 7.6$	$18/5 = 3.6$

Rendement : $5/20 = 0.25$ processus/unité de temps.

2.6 SRT (Shortest Remaining Time)

Variante préemptive du SJF : à chaque arrivée d'un nouveau processus, on compare son temps de traitement avec le temps restant du processus courant. Si le nouveau processus est plus court, il prend le CPU.

- $t = 0$: P1(3) seul $\rightarrow$ P1 exécute.
- $t = 2$: P2 arrive (6) $\rightarrow$ P1 reste 1 < 6, P1 continue.
- $t = 3$: **P1 finit**. P2(6) seul disponible.
- $t = 4$: P3 arrive (4) $\rightarrow$ P2 reste 5 > 4 $\rightarrow$ **préemption**, P3 prend le CPU.
- $t = 6$: P4 arrive (5) $\rightarrow$ P3 reste 2 < 5, P3 continue.
- $t = 8$: P5 arrive (2), P3 finit exactement $\rightarrow$ **P3 finit à $t = 8$** . Disponibles : P2(5), P4(5), P5(2) $\rightarrow$ P5.
- $t = 10$: **P5 finit**. P2(5) et P4(5) égaux $\rightarrow$ P2 (arrivé en premier).
- $t = 15$: **P2 finit**. P4 $\rightarrow$ **P4 finit à $t = 20$** .



Processus	Fin	Rotation	Attente
P1	3	3	0
P2	15	13	7
P3	8	4	0
P4	20	14	9
P5	10	2	0
Moyenne		$36/5 = 7.2$	$17/5 = 3.4$

Rendement : $5/20 = 0.25$ processus/unité de temps.

3 Exercice 3 : Algorithmes d'ordonnancement — 4 processus

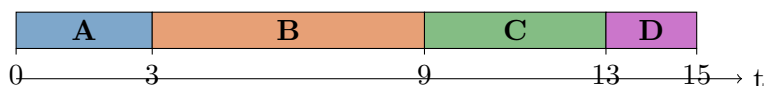
3.1 Données

Processus	Date d'arrivée	Temps de traitement
A	0	3
B	1	6
C	4	4
D	6	2

La durée totale d'exécution est $3 + 6 + 4 + 2 = 15$ unités de temps.

3.2 FCFS

Ordre d'arrivée : A, B, C, D.

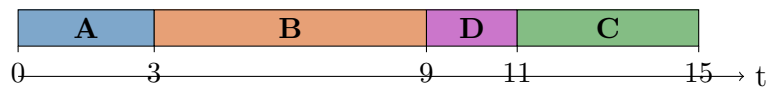


Processus	Fin	Rotation	Attente
A	3	3	0
B	9	8	2
C	13	9	5
D	15	9	7
Moyenne		$29/4 = 7.25$	$14/4 = 3.5$

Rendement : $4/15 \approx 0.267$ processus/unité de temps.

3.3 SJF sans préemption

- $t = 0$: seul A $\rightarrow$ A finit à $t = 3$.
- $t = 3$: seul B disponible (C pas encore) $\rightarrow$ B finit à $t = 9$.
- $t = 9$: C(4) et D(2) disponibles $\rightarrow$ D (plus court), finit à $t = 11$.
- $t = 11$: C $\rightarrow$ finit à $t = 15$.

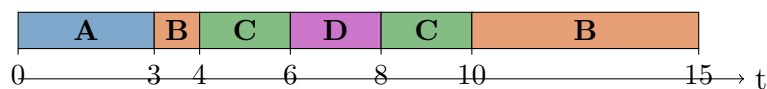


Processus	Fin	Rotation	Attente
A	3	3	0
B	9	8	2
D	11	5	3
C	15	11	7
Moyenne		$27/4 = 6.75$	$12/4 = 3$

Rendement : $4/15 \approx 0.267$ processus/unité de temps.

3.4 SJF avec préemption (SRT)

- $t = 0$: A(3) seul $\rightarrow$ A.
- $t = 1$: B arrive (6) $\rightarrow$ A reste $2 < 6$, A continue.
- $t = 3$: **A finit**. B(6) seul.
- $t = 4$: C arrive (4) $\rightarrow$ B reste $5 > 4 \rightarrow$ **préemption**, C prend le CPU.
- $t = 6$: D arrive (2), C reste $2 = 2 \rightarrow$ D prend le CPU (durée plus courte ou arrivée).
- $t = 8$: **D finit**. C reste 2 $\rightarrow$ C.
- $t = 10$: **C finit**. B reste 5 $\rightarrow$ B.
- $t = 15$: **B finit**.

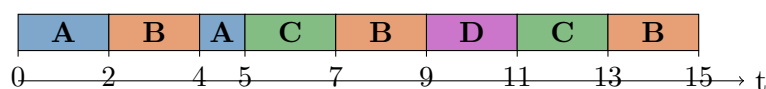


Processus	Fin	Rotation	Attente
A	3	3	0
B	15	14	8
C	10	6	2
D	8	2	0
Moyenne		$25/4 = 6.25$	$10/4 = 2.5$

Rendement : $4/15 \approx 0.267$ processus/unité de temps.

3.5 Round Robin — quantum = 2

- $t = 0 \rightarrow 2$: A(reste 1) — A exécute 2 unités, reste 1.
- $t = 2 \rightarrow 4$: B arrive à $t = 1 \rightarrow \text{file}=[A,B]$. B exécute, reste 4. A arrive à $t=4$: $[A,C,B]$
- $t = 4 \rightarrow 5$: A exécute son quantum restant de 1 $\rightarrow$ **A finit à $t = 5$** .
- $t = 5 \rightarrow 7$: $\text{file}=[C,B] \rightarrow$ C exécute, reste 2.
- $t = 6$: D arrive $\rightarrow$ mis en $\text{file}=[B,D,C]$.
- $t = 7 \rightarrow 9$: B exécute, reste 2.
- $t = 9 \rightarrow 11$: D exécute $\rightarrow$ **D finit à $t = 11$** .
- $t = 11 \rightarrow 13$: C exécute $\rightarrow$ **C finit à $t = 13$** .
- $t = 13 \rightarrow 15$: B exécute $\rightarrow$ **B finit à $t = 15$** .



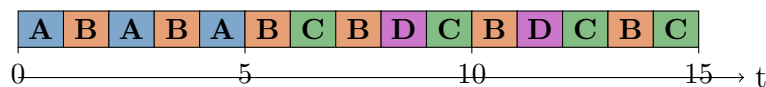
Processus	Fin	Rotation	Attente
A	5	5	2
B	15	14	8
C	13	9	5
D	11	5	3
Moyenne		$33/4 = 8.25$	$18/4 = 4.5$

Rendement : $4/15 \approx 0.267$ processus/unité de temps.

3.6 Round Robin — quantum = 1

L'exécution unité par unité donne la séquence suivante :

$t = 0 \rightarrow 1$: A(2), $t = 1 \rightarrow 2$: B(5), $t = 2 \rightarrow 3$: A(1), $t = 3 \rightarrow 4$: B(4), $t = 4 \rightarrow 5$: A $\rightarrow$ **fin A t=5**, $t = 5 \rightarrow 6$: B(3), $t = 6 \rightarrow 7$: C(3) [D arrive], $t = 7 \rightarrow 8$: B(2), $t = 8 \rightarrow 9$: D(1), $t = 9 \rightarrow 10$: C(2), $t = 10 \rightarrow 11$: B(1), $t = 11 \rightarrow 12$: D $\rightarrow$ **fin D t=12**, $t = 12 \rightarrow 13$: C(1), $t = 13 \rightarrow 14$: B $\rightarrow$ **fin B t=14**, $t = 14 \rightarrow 15$: C $\rightarrow$ **fin C t=15**.



Processus	Fin	Rotation	Attente
A	5	5	2
B	14	13	7
C	15	11	7
D	12	6	4
Moyenne		$35/4 = 8.75$	$20/4 = 5$

Rendement : $4/15 \approx 0.267$ processus/unité de temps.

4 Exercice 4 : Ordonnancement par priorités

4.1 Données

Un numéro de priorité élevé correspond à une priorité plus importante.

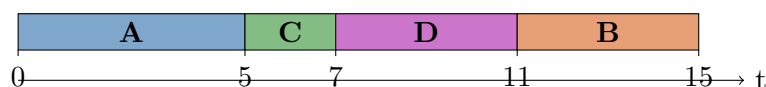
Processus	Date d'arrivée	Temps de traitement	Priorité
A	0	5	4
B	2	4	2
C	2	2	6
D	4	4	3

La durée totale d'exécution est $5 + 4 + 2 + 4 = 15$ unités de temps.

4.2 Ordonnancement par priorités — sans préemption

Une fois qu'un processus a le CPU, il le conserve jusqu'à sa fin. À chaque libération du CPU, on choisit le processus disponible de plus haute priorité.

- $t = 0$: seul A disponible (priorité 4) → A s'exécute jusqu'à la fin.
- $t = 5$: **A finit**. Disponibles : B(2), C(6), D(3) → C priorité max (6) → C.
- $t = 7$: **C finit**. Disponibles : B(2), D(3) → D priorité 3 > B priorité 2 → D.
- $t = 11$: **D finit**. B → **B finit à $t = 15$** .



Processus	Fin	Rotation	Attente
A	5	$5 - 0 = 5$	0
C	7	$7 - 2 = 5$	3
D	11	$11 - 4 = 7$	3
B	15	$15 - 2 = 13$	9
Moyenne		$30/4 = 7.5$	$15/4 = 3.75$

Rendement : $4/15 \approx 0.267$ processus/unité de temps.

4.3 Ordonnancement par priorités — avec préemption

À chaque arrivée d'un nouveau processus, si sa priorité est supérieure au processus courant, il y a préemption.

- $t = 0$: A(priorité 4) seul → A exécute.
- $t = 2$: B(2) et C(6) arrivent → C priorité 6 > A priorité 4 → **préemption**, C prend le CPU. A reste = 3.
- $t = 4$: C finit (durée 2). **C finit à $t = 4$** . D arrive (priorité 3). Disponibles : A(reste 3, priorité 4), B(priorité 2), D(priorité 3). A a la plus haute priorité → A reprend.
- $t = 7$: **A finit** (avait exécuté 2 unités avant préemption + 3 unités après). Disponibles : B(2), D(3) → D (priorité 3 > 2) → D.
- $t = 11$: **D finit**. B → **B finit à $t = 15$** .



Processus	Fin	Rotation	Attente
A	7	$7 - 0 = 7$	2
B	15	$15 - 2 = 13$	9
C	4	$4 - 2 = 2$	0
D	11	$11 - 4 = 7$	3
Moyenne		$29/4 = 7.25$	$14/4 = 3.5$

Rendement : $4/15 \approx 0.267$ processus/unité de temps.

Conclusion

Ce TP3 a permis d'étudier et de comparer les principales politiques d'ordonnancement de processus. L'analyse des différents algorithmes appliqués aux mêmes jeux de données met en évidence des différences significatives de performance selon le critère retenu.

L'algorithme FCFS est le plus simple à implémenter mais peut engendrer des temps d'attente élevés pour les processus courts qui arrivent après un processus long (effet convoi). Le SJF sans préemption améliore les temps moyens de rotation et d'attente, mais au détriment des processus longs qui peuvent souffrir de famine. Le SRT, variante préemptive du SJF, donne généralement les meilleurs temps moyens de rotation et d'attente, au prix d'une plus grande complexité de gestion. Le Round Robin offre une bonne équité entre les processus mais génère plus de commutations de contexte, notamment avec un petit quantum. Enfin, l'ordonnancement par priorités permet de différencier l'importance des processus, avec préemption pour une réactivité accrue.

Le choix de l'algorithme d'ordonnancement dépend donc du contexte d'utilisation : systèmes temps réel, systèmes interactifs ou systèmes par lots.